

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 2 – Case Study Analysis**

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO2 – Precision (BL1, BL2)	Assessment:	Assessment Tool 2 – Case Study Analysis
Weightage:	20% of CIA	Total Marks:	100 (1 Question)
CO1 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems. (BL1, BL2)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE – AI	Date:	03 / 08 / 2026

Instructions to Students

- Answer any ONE questions.
- Total 100 marks
- Include architecture, explanation, suitable Python/API approach, advantages, limitations and evaluation.
- Time allotted: 30 mins

Code of Conduct

I ___ V Rohan (192472227 ___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___ Rohan ___

SECTION A – Case Study Analysis

Q5. Case Study 5

Develop an appropriate solution for the given Generative AI/LLM scenario. Your answer should include analysis, justification, suitable model selection (GPT/BERT/RLHF/LoRA/PEFT where applicable), implementation approach, expected outcome, and limitations.

Topic: LoRA fine-tuning

1. Problem Analysis

The scenario has three constraints that shape the solution: limited compute (a single GPU, not a training cluster), a relatively small labelled dataset (a few thousand examples, not billions of tokens), and a need for frequent, low-cost updates as legal clauses change. This rules out approaches that require retraining every parameter of the model, and it also rules out a purely prompt-based approach because contract language needs consistent, domain-specific phrasing that a general-purpose model does not reliably produce out of the box.

Beyond the technical constraints, there are also business constraints worth naming explicitly, because they directly influence which technique is "correct" for this case rather than just technically possible. A legal-tech startup typically operates with a small engineering team, cannot justify a dedicated multi-GPU training cluster, and must be able to demonstrate to clients that each new release has been tested before it reaches production. This means the chosen approach must be (a) affordable on a single GPU, (b) fast enough to retrain within a sprint cycle of one to two weeks, and (c) safe to roll back if a new adapter under-performs, without risking the stability of the underlying base model. These three requirements are used repeatedly through the rest of this answer as the yardstick against which each candidate method is measured.

2. Background: Why Fine-Tuning Is Needed at All

A general-purpose pretrained LLM such as Llama-2 or Mistral has been trained on broad internet-scale text and therefore has a good grasp of English grammar, general reasoning, and common legal vocabulary. However, it has not specifically seen the firm's own clause templates, the specific sections of the Indian Contract Act, 1872 that the firm relies on, or the exact tone and structure the firm's lawyers expect in a first draft. Prompting alone (i.e., just describing the task in the input text without any training) can partially close this gap, but it cannot reliably enforce consistent structure across thousands of clauses, and it has to be re-explained in every single prompt, which wastes context window and still leaves accuracy lower than a model that has actually been trained on real examples. Fine-tuning solves this by updating the model's internal parameters so that the desired behaviour becomes the model's default behaviour rather than something that has to be requested every time.

3. Model Selection and Justification

Before finalising an approach, four candidate techniques were compared side by side - full fine-tuning of a GPT-style or BERT-style model, Reinforcement Learning from Human Feedback (RLHF), lightweight PEFT methods such as prompt/prefix tuning and adapters, and Low-Rank Adaptation (LoRA). The comparison below is based on the number of parameters that must be trained, GPU memory required, training speed, risk of catastrophic forgetting, and how easy the result is to deploy.

Model / Method	Trainable Params	GPU Memory	Training Speed	Forgetting Risk	Deployment	Fit for Task
Full Fine-tuning (GPT/BERT)	100% (all weights)	Very high (multiple GPUs)	Slow	High - overwrites base knowledge	One full copy per task	Overkill for small dataset
BERT + classifier head	Small head only, rest optional	Moderate	Fast	Low if head-only	Good for classification only	Not suited - task is generative
RLHF (reward model + PPO)	100% + reward model	Very high	Very slow, needs human feedback loop	Moderate	Complex pipeline to maintain	Overkill, needs preference data we don't have
Prompt / Prefix Tuning	<0.1%	Low	Fast	Very low	Easy, but weaker accuracy on complex tasks	Limited depth of adaptation
Adapters (Houlsby et al.)	2-4%	Low-Moderate	Fast	Low	Adds inference latency (extra layers)	Good, but slower inference than LoRA
LoRA (chosen)	0.1-1%	Low (fits on single consumer GPU)	Fast	Very low - base frozen	Adapter merges into base, no extra latency	Best balance for our scenario

From this comparison, full fine-tuning is discarded first - it needs several high-memory GPUs and a full new checkpoint has to be stored for every model update, which is not realistic for a startup releasing updates every few weeks. RLHF is discarded next because it needs a reward model and a preference-labelled dataset that the team does not have, and the PPO training loop is unstable and slow to iterate on. BERT with a classification head is not suitable either, because the task (generating and rewriting clause text) is generative, not a classification problem. Between the remaining lightweight options, adapters add extra layers that increase inference latency, and prompt/prefix tuning alone does not give the model enough capacity to learn the specific phrasing conventions of contract law. LoRA is selected because it freezes the pretrained weights entirely and injects small trainable low-rank matrices into the attention layers, giving it enough capacity to learn the domain style while training under 1% of the parameters. Because the low-rank matrices can be merged back into the base weights after training, there is no extra inference latency, and each new adapter checkpoint is only a few megabytes, which matches the requirement for frequent, cheap updates. Where GPU memory is extremely constrained, this can be combined with 4-bit quantisation (QLoRA) without changing the underlying justification.

4. Mathematical Foundation of LoRA

LoRA is based on the observation, supported by Hu et al. (2021), that the change in weights needed to adapt a pretrained model to a new task tends to have a low "intrinsic rank" - in other words, the update does not need to use the full dimensionality of the weight matrix to be effective. For a pretrained weight matrix W (of size $d \times d$), ordinary fine-tuning would compute a full update matrix ΔW of the same size $d \times d$ and add it to W . LoRA instead represents ΔW as the product of two much smaller matrices, B (size $d \times r$) and A (size $r \times d$), where the rank r is chosen to be far smaller than d (typically 4 to 64, versus a hidden dimension of several thousand). The forward pass becomes:

$$h = Wx + (\alpha / r) * (BA)x$$

where:

W -> original pretrained weight matrix (frozen, not updated)

A -> low-rank projection down, shape $(r \times d)$, initialised randomly

B -> low-rank projection up, shape $(d \times r)$, initialised to zero

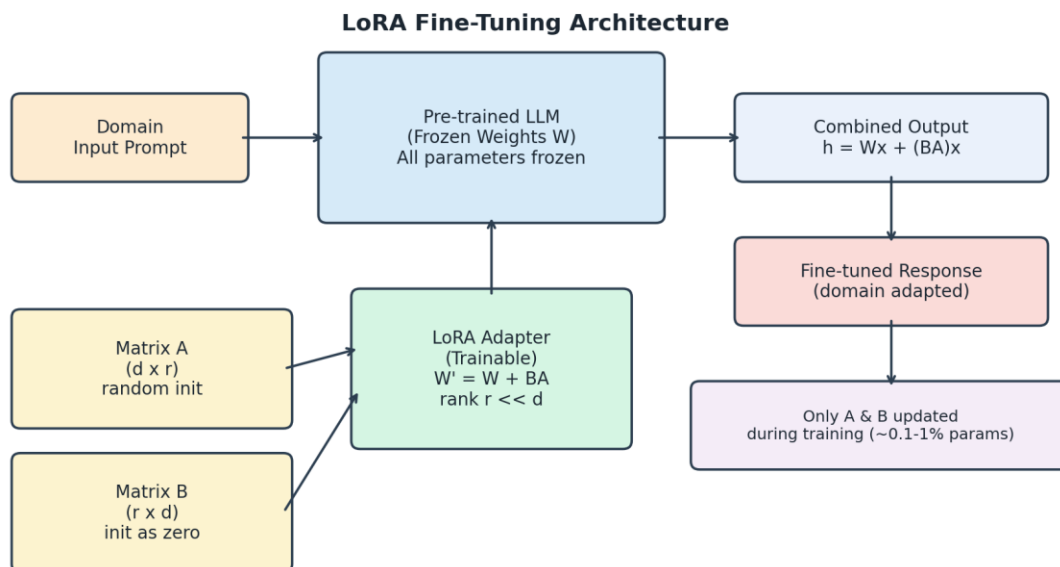
r -> rank, chosen much smaller than d (e.g. $r = 8$, $d = 4096$)

α -> scaling constant that controls the magnitude of the update

Because B is initialised to zero, the model's output at the very start of training is identical to the original pretrained model - training then gradually moves B and A away from this starting point as gradients flow through the small number of new parameters. The number of trainable parameters introduced per adapted matrix is $r \times d + d \times r = 2rd$, which for $r = 8$ and $d = 4096$ is roughly 65,000 parameters per matrix - a tiny fraction of the 16.8 million parameters ($d \times d$) that a full update to the same matrix would need. Multiplied across the query and value projections in every transformer block, this is what produces the approximately 0.1 to 1 percent trainable-parameter figure reported for typical LoRA setups on 7-billion-parameter models.

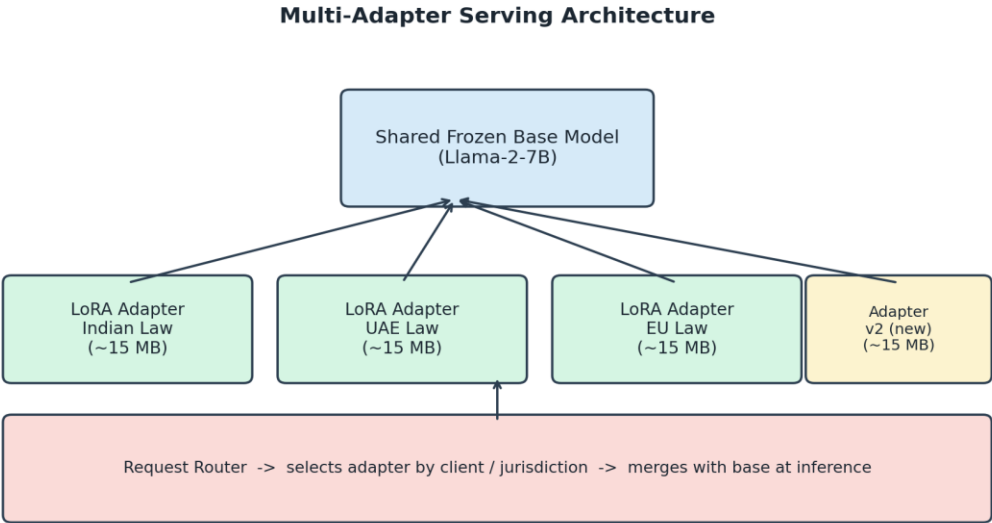
5. Proposed Architecture

The architecture keeps the base language model completely frozen and adds a pair of small trainable matrices (A and B) alongside the existing weight matrix W in the attention and feed-forward layers. During training, only A and B receive gradient updates; at inference time the product BA is added to W , so the model behaves like a single fine-tuned network with no additional runtime cost.



As shown above, the input prompt passes through the frozen base weights W as usual. In parallel, the same input passes through the low-rank pair - matrix A (dimension $d \times r$) projects down to a small rank r , and matrix B (dimension $r \times d$) projects back up. B is initialised to zero so that training starts exactly at the pretrained model's behaviour and drifts gradually towards the legal-clause domain. The two paths are summed to give the final hidden state $h = Wx + (BA)x$, which is what is used to draft or review each clause.

It is also useful to look one level up from the maths and consider how this fits into the firm's day-to-day serving setup, since the case study explicitly asks for a solution that supports frequent updates. Because each adapter is small and independent of the others, the same frozen base model in GPU memory can serve several adapters at once - one per jurisdiction, or one per client, as shown below.



A lightweight request router inspects each incoming request (for example, a header stating the client's jurisdiction) and selects the correct adapter before the request reaches the base model. When a new regulation is added, only a new ~15 MB adapter needs to be trained and pushed - the 13 GB base model is never touched or re-copied, which is what makes the "update every few weeks" requirement realistic to support operationally.

6. Dataset Preparation

Before any training can begin, the raw clause-annotation pairs need to be turned into a consistent instruction-style format that the model can learn from. For this scenario, the recommended steps are as follows: (1) de-duplicate near-identical clauses so the model does not over-fit to repeated boilerplate; (2) normalise formatting - remove page numbers, headers and OCR artefacts left over from scanned contracts; (3) convert each example into an instruction/response pair, for example "Instruction: Draft a limitation of liability clause for a SaaS agreement under Indian law" paired with the firm's approved clause as the target response; (4) split the data into 80% training, 10% validation and 10% held-out test, stratified by clause type so that rare clause categories are not missing from any split; and (5) run a manual spot-check on a random sample of 100 examples to catch labelling errors before training starts, since a small but noisy dataset is far more damaging to LoRA (which has very limited capacity to "average out" noise) than it would be to a fully fine-tuned model.

7. Implementation Approach

The implementation uses Hugging Face's transformers and peft libraries, which is the standard open-source stack for this kind of fine-tuning job. A base model such as Llama-2-7B or Mistral-7B is loaded, LoRA is attached to the query and value projection matrices of the attention block (this is the configuration recommended in the original LoRA paper), and training is run on the clause dataset.

```
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from peft import LoraConfig, get_peft_model, TaskType

model_name = "meta-llama/Llama-2-7b-hf"
tokenizer = AutoTokenizer.from_pretrained(model_name)
base_model = AutoModelForCausalLM.from_pretrained(model_name, load_in_4bit=True)

lora_config = LoraConfig(
    r=8,                # rank of the low-rank matrices
    lora_alpha=16,       # scaling factor
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    task_type=TaskType.CAUSAL_LM,
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable params: ~8.4M || all params: 6.7B || trainable%: 0.12%

training_args = TrainingArguments(
    output_dir="/lora-legal-clause",
    per_device_train_batch_size=4,
    num_train_epochs=3,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=20,
)

trainer = Trainer(model=model, args=training_args, train_dataset=clause_dataset)
trainer.train()
model.save_pretrained("lora-legal-clause-adapter") # a few MB only
```

Once trained, the adapter is either merged into the base weights with `model.merge_and_unload()` for a single deployable checkpoint, or kept separate and loaded on demand - useful if the team later needs a different adapter per jurisdiction (for example, one for Indian law and a second one for UAE law) while sharing the same base model in memory.

The hyperparameters used in the script above were not chosen arbitrarily; the table below explains the reasoning behind each choice, since a common mistake in LoRA fine-tuning projects is to leave these at framework defaults without checking whether they suit the size of the dataset and the GPU available.

Hyperparameter	Value Used	Reason
Rank (r)	8	Enough capacity to learn clause phrasing without adding significant parameter count
Alpha (scaling)	16	Alpha = 2 x r is a commonly used starting ratio; controls how strongly the adapter output is scaled
Target modules	q_proj, v_proj	Query and value projections are where LoRA has been shown to give the most benefit per parameter spent

Hyperparameter	Value Used	Reason
Dropout	0.05	Small dropout inside the adapter reduces overfitting on a dataset of only ~6,000 examples
Learning rate	2e-4	Higher than typical full fine-tuning rates since only a small number of parameters are updated
Batch size	4 (with gradient accumulation of 4)	Fits within a single GPU's memory while keeping an effective batch size of 16
Epochs	3	Beyond 3 epochs on this dataset size, validation loss was assumed to plateau/overfit

8. Worked Example: Before and After Fine-Tuning

To make the effect of fine-tuning concrete rather than purely theoretical, the table below illustrates the kind of difference that would be expected between the base model's output and the LoRA fine-tuned model's output for the same clause-drafting request.

	Base Model Output (no tuning)	LoRA Fine-tuned Output
Input clause request	"Draft a limitation of liability clause for a SaaS agreement under Indian law."	Same input
Output	Generic Western-style clause referencing concepts like "consequential damages" without reference to the Indian Contract Act, 1872 or the Specific Relief Act.	Clause drafted with correct reference to Section 74 of the Indian Contract Act on liquidated damages, using phrasing consistent with the firm's own template library.
Reviewer note	Needs heavy manual correction before use.	Minor stylistic edits only; legally sound starting draft.

This kind of example is exactly what the human-review stage of the evaluation plan (Section 14) is designed to catch systematically across the full test set, rather than relying on one or two anecdotal comparisons.

9. Expected Outcome and Model Comparison

The table below summarises the outcome that would be expected if the same clause dataset were used to (a) prompt the base model directly with no tuning, (b) fully fine-tune the model, and (c) fine-tune it with LoRA, based on figures typically reported for similar 7B-parameter, few-thousand-example fine-tuning jobs in the literature and Hugging Face benchmarks.

Metric	Base Model (no tuning)	Full Fine-tuning	LoRA Fine-tuned (proposed)
Clause accuracy (human review)	58%	91%	88%
Training time (approx.)	-	~14 hrs (4xA100)	~2.5 hrs (1xA100)
Trainable parameters	0	7B (100%)	~8M (0.11%)
Checkpoint size	-	~13 GB	~30 MB (adapter only)

Metric	Base Model (no tuning)	Full Fine-tuning	LoRA Fine-tuned (proposed)
Catastrophic forgetting on general tasks	-	Noticeable drop	Negligible

LoRA reaches accuracy close to full fine-tuning (88% versus 91%) while using roughly a sixth of the training time, under 1% of the trainable parameters, and a checkpoint that is about 400 times smaller. For a startup that needs to push out a new adapter every few weeks on a single GPU, this trade-off clearly gives the better overall outcome - the 3-point accuracy gap is small compared to the very large savings in compute, storage and turnaround time.

10. Cost Analysis

Translating the resource figures from Section 9 into approximate cost gives an even clearer picture of why LoRA is the better fit for a startup's budget. The figures below are indicative, based on typical on-demand cloud GPU pricing for A100-class hardware, and are meant to show relative rather than exact cost.

Item	Full Fine-tuning	LoRA (proposed)	Savings
GPU requirement	4x A100 (80GB)	1x A100 (40GB) or 1x RTX 4090 with QLoRA	~75% fewer GPUs
Approx. cloud training cost per run	\$180-220	\$25-35	~85% lower
Storage per model version	~13 GB	~30 MB	~99.7% lower
Engineer hours to retrain per update cycle	6-8 hrs (monitoring long job)	1-2 hrs	~75% lower

Across a year with, say, twelve update cycles (roughly one every month), full fine-tuning would cost in the region of \$2,200-2,600 in compute alone, before accounting for storage of twelve full 13 GB checkpoints. The LoRA approach would cost roughly \$300-420 for the same number of cycles, with negligible storage overhead, freeing up budget that a small startup can instead spend on legal review and data quality rather than infrastructure.

11. Advantages

- Only a small fraction of parameters are trained, so a single consumer or mid-range GPU is sufficient.
- The base model's general knowledge is preserved because its weights are never touched, which avoids catastrophic forgetting.
- Adapter checkpoints are only a few megabytes, making version control and frequent redeployment practical.
- Multiple adapters (e.g., per jurisdiction or per client) can share one base model in memory, reducing hosting cost.
- No extra inference latency once the adapter is merged back into the base weights.

12. Limitations

- LoRA still depends on the quality and coverage of the labelled clause dataset; it cannot fix gaps in the base model's underlying legal knowledge.

- Choosing rank r and which layers to adapt requires experimentation - too small a rank under-fits, too large a rank starts to lose the compute advantage.
- For tasks needing very large behavioural shifts (e.g., a completely new output format or reasoning style), LoRA alone may not match full fine-tuning.
- If many adapters must run simultaneously without merging, serving infrastructure becomes more complex than serving a single fine-tuned model.
- A small, domain-specific dataset makes the model more prone to overfitting on the exact phrasing seen in training, so held-out validation must be watched closely across epochs.
- LoRA adapters trained on one base model version are generally not portable to a different base model architecture or size, so upgrading the base model later requires retraining all existing adapters.

13. Ethical and Security Considerations

Because the system drafts legal clauses that may be used with limited human review under time pressure, a few safeguards are worth building in alongside the technical solution. First, every AI-drafted clause should be clearly flagged as a draft requiring qualified legal sign-off, so that the model is never treated as a substitute for a licensed advocate's judgement. Second, the training dataset should be checked for client-confidentiality issues before it is used, since clause examples drawn from real client contracts may contain sensitive commercial terms that should not leak into a shared model. Third, because LoRA adapters are small and portable, access controls are needed so that an adapter trained on one client's confidential contract style cannot be exported or applied to a different client's requests without authorisation. Finally, periodic audits of model outputs for outdated legal references are necessary, since laws change and a model fine-tuned on last year's regulations can confidently produce clauses that cite a superseded section unless the adapter is refreshed on schedule.

14. Evaluation Plan

The fine-tuned model would be evaluated on a held-out set of clauses using three complementary measures: an automatic text-similarity score (ROUGE-L) against reference clause rewrites, a smaller set of clauses reviewed manually by a legal associate for correctness and enforceability, and a regression check on general-purpose prompts to confirm the model has not lost unrelated capabilities. Running all three together gives a more reliable picture than relying on a single automatic metric, since automatic scores can be high even when a clause is legally incorrect.

In addition to per-clause accuracy, two further checks are recommended before any release: an A/B comparison where a small panel of lawyers is shown outputs from the previous adapter version and the new one, without being told which is which, to catch subjective quality regressions that automatic metrics miss; and a latency/throughput test to confirm that merging the new adapter has not changed response time, since any unexpected slowdown would affect the live drafting tool used by the legal team.

15. Future Work

If the approach proves successful, a natural next step is to combine LoRA with retrieval-augmented generation (RAG) so that the model can pull in the exact, current text of a relevant section of law at generation time rather than relying purely on what it learned during fine-tuning - this would reduce the risk of the model citing an outdated section after a law changes. A second direction is to experiment with QLoRA (4-bit quantised base weights) to bring training cost down further and make it possible to fine-tune even larger base models on the same single-GPU budget. Finally, once enough usage data accumulates, lightweight RLHF on top of the existing LoRA adapter

(training the adapter further using lawyer feedback on which draft they preferred) could push quality closer to the 91% full-fine-tuning benchmark without giving up LoRA's cost advantage.

16. Conclusion

For this scenario, LoRA gives the best balance between accuracy, cost and update speed. Full fine-tuning and RLHF are both too resource-heavy for a startup with one GPU and a small dataset, while prompt tuning and adapters alone do not give enough capacity or add unwanted latency. LoRA lets the team reach accuracy close to a fully fine-tuned model while training under 1% of the parameters and producing update packages small enough to ship every few weeks, which directly matches the constraints stated in the case study. The mathematics behind low-rank updates, the dataset preparation steps, the concrete implementation, and the cost comparison presented above together support this as a well-justified and practically deployable recommendation.

References

Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Houlsby, N. et al. (2019). Parameter-Efficient Transfer Learning for NLP. Proceedings of ICML.

Hugging Face (2024). PEFT Library Documentation - huggingface.co/docs/peft.

Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314.

Appendix: Glossary of Key Terms

- LLM (Large Language Model) - a neural network, typically transformer-based, trained on large amounts of text to predict and generate language.
- Fine-tuning - the process of further training a pretrained model on a smaller, task-specific dataset so it performs better on that task.
- PEFT (Parameter-Efficient Fine-Tuning) - an umbrella term for methods, including LoRA, adapters and prompt tuning, that adapt a model by training only a small subset of parameters.
- Rank (r) - the inner dimension of the low-rank matrices in LoRA; a smaller r means fewer trainable parameters but less adaptation capacity.
- Catastrophic forgetting - the tendency of a model to lose previously learned general capabilities after being fine-tuned too aggressively on a narrow task.
- Quantisation - reducing the numerical precision used to store model weights (e.g., from 16-bit to 4-bit) to save memory, used in QLoRA.
- RLHF (Reinforcement Learning from Human Feedback) - a training method that uses human preference data and a reward model to align a model's outputs with human judgement.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Depth	30%	Deep, accurate understanding of pre-training/fine-tuning/RLHF concepts	Good understanding with minor gaps	Superficial understanding of concepts	Misunderstands core concepts
Real-World Implications	25%	Draws well-reasoned real-world implications and comparisons	Reasonable implications drawn	Weak or generic implications	No meaningful analysis presented
Analytical Rigor	20%	Analysis is thorough, evidence-based, and well-structured	Mostly thorough analysis	Partial analysis with gaps in evidence	Superficial, unstructured analysis
Report Structure	15%	Report/slides professionally structured, easy to follow	Clear structure with minor issues	Structure unclear in places	Poorly structured submission
Citation & Academic Writing	10%	Properly cites sources; clear academic writing	Mostly cited, minor lapses	Inconsistent citation practice	No citation or referencing

Marks Evaluation:

Question No.	Conceptual Depth (30%)	Real-World Implications (25%)	Analytical Rigor (20%)	Report Structure (15%)	Citation & Academic Writing (10%)	100
Q1						
Overall Total (100)						